

Avalei e responda as questões:

Abaixo está a lista completa com **30 exercícios** organizados de forma gradual para você implementar durante a aula.



Lista de Exercícios Práticos: Ponteiros e Vetores em C

Bloco 1: Fundamentos de Memória, Operadores e sizeof

1. Tamanho dos Tipos de Dados:

Escreva um programa em C que utilize o operador sizeof para imprimir o tamanho em bytes dos seguintes tipos: char, short int, int, long int, float, double e long double. Imprima também o endereço de cada variável utilizada.

2. Manipulação Básica de Endereços:

Declare três variáveis (int x, float y, char z). Atribua valores a elas e imprima seus valores e seus respectivos endereços de memória em hexadecimal utilizando o especificador %p (ou %X) e o operador &.

3. Ponteiro Simples e Desreferenciação(acesso ao ponteiro & ou *) :

Declare uma variável inteira a = 15 e um ponteiro int *pa. Faça pa apontar para a. Altere o valor de a para 50 utilizando apenas o ponteiro pa e exiba o novo valor de a.

4. Comparação de Endereços:

Escreva um programa que declare duas variáveis inteiras e compare seus endereços na memória. O programa deve exibir qual das duas variáveis está alocada no maior endereço de memória.

5. Operações Aritméticas via Ponteiros:

Crie um programa que leia dois inteiros, num1 e num2. Declare ponteiros para ambos e, através dos ponteiros, calcule e exiba a soma, a diferença e o produto entre eles.

Bloco 2: Alocação Dinâmica com malloc e free

6. Alocação Dinâmica de Inteiro:

Crie um ponteiro para inteiro iniciado com NULL. Aloque memória dinamicamente usando malloc para armazenar um valor inteiro. Verifique se a alocação foi bem-sucedida, atribua um valor lido pelo usuário, imprima-o e libere a memória com free.

7. Soma Dinâmica de Idades:

(Baseado no Exercício 2 da aula) Leia duas variáveis inteiras A e B (idades). Aloque memória dinamicamente para um inteiro apontado por *ps. Armazene a soma $A + B$ nesse espaço. Imprima:

- O resultado da soma;
- O endereço armazenado em ps e o endereço do próprio ponteiro &ps;
- Os endereços das variáveis A e B.

8. Alocação Dinâmica com double:

(Baseado no Exercício 3 da aula) Leia dois valores double A e B (pesos). Aloque memória dinamicamente para armazenar a soma $A + B$ apontada por *ps. Em seguida, declare uma variável double C, copie o valor referenciado por *ps para C e exiba C. Libere a memória.

9. Atribuição Direta e Modificação Dinâmica:

(Baseado no Exercício 4 da aula) A partir do exercício anterior, crie um segundo ponteiro *qs e faça-o receber o mesmo endereço de *ps.

- Somar 100 ao valor referenciado por *qs e exibir *ps e *qs;
- Apontar qs para a variável B e subtrair 10 do seu valor usando *qs;
- Exibir os valores finais.

10. Vetor Dinâmico Simples:

Solicite ao usuário o tamanho de um vetor de inteiros \$N\$. Aloque dinamicamente memória para \$N\$ inteiros usando malloc, preencha-o com números informados pelo usuário, exiba-os e libere a memória.

Bloco 3: Passagem de Parâmetros por Referência

11. Troca de Valores (Passagem por Referência):

Escreva uma função void troca(int *a, int *b) que receba dois ponteiros para inteiros e troque os valores das variáveis originais. Escreva uma função main para testar.

12. Soma via Referência:

(Baseado no Exercício 4 da aula) Crie a função void somaAB(int a, int b, int *pab) que armazene no ponteiro *pab o resultado da soma de a e b. Na main, exiba o valor final da variável associada.

13.Divisão Real via Referência:

(Baseado no Exercício 5 da aula) Crie a função void `dividaAB(int *pa, int *pb, double *pab)` que receba o endereço de dois inteiros, realize a divisão entre eles (convertendo para double) e atribua o resultado ao ponteiro `*pab`.

14.Conversor de Minutos para Horas:

(Baseado no Exercício 12 da aula) Escreva a função void `converte(int mnts, int *h, int *m)`. A função deve atribuir aos endereços de `h` e `m` as horas (`mnts / 60`) e minutos restantes (`mnts % 60`). Teste na main.

15.Estatísticas de Dois Inteiros:

Crie uma função void `operacoes(int a, int b, int *soma, int *diff, int *prod)` que receba dois inteiros e retorne, por referência, a soma, a diferença e o produto entre eles.

Bloco 4: Vetores e Aritmética de Ponteiros

16.Impressão de Vetor com Aritmética de Ponteiros:

Crie uma função void `imprimeVetorPonteiro(int *p, int n)` que imprima todos os elementos de um vetor de inteiros utilizando navegação por ponteiro `*(p + i)` em vez do operador de subscrito `p[i]`.

17.Menor Elemento do Vetor:

(Baseado no Exercício 6 da aula) Escreva a função void `menorVetor(int *vt, int n, int *menor)` que identifique o menor valor dentro do vetor `vt` e o deposite na variável passada por referência `menor`.

18.Maior Elemento do Vetor:

(Baseado no Exercício 7 da aula) Escreva a função void `maiorVetor(int *vt, int n, int *maior)` que encontre o maior elemento do vetor e armazene na variável apontada por `maior`.

19. **Menor e Maior Combinados:**

(Baseado no Exercício 8 da aula) Escreva a função void menorMaiorVetor(int *vt, int n, int *menor, int *maior) que utilize internamente as funções criadas nos exercícios 17 e 18 para obter o menor e maior elemento do vetor simultaneamente.

20. **Retorno de Endereço do Menor Elemento:**

(Baseado no Exercício 9 da aula) Crie a função int* menorVetorPonteiro(int *vt, int n) que retorne diretamente o **endereço** do menor elemento do vetor.

21. **Retorno de Endereço do Maior Elemento:**

(Baseado no Exercício 10 da aula) Crie a função int* maiorVetorPonteiro(int *vt, int n) que retorne o **endereço** do maior elemento do vetor.

22. **Inversão de Vetor via Ponteiros:**

Escreva uma função void inverteVetor(int *v, int n) que inverta os elementos de um vetor no próprio local (in-place) utilizando dois ponteiros (um apontando para o início e outro para o fim).

Bloco 5: Processamento de Strings via Ponteiros

23. **Contador de Vogais:**

(Baseado no Exercício 15 da aula) Escreva uma função int contaVogais(char *str) que percorra uma string utilizando ponteiros e retorne a quantidade total de vogais.

24. **Contador de Consoantes:**

(Baseado no Exercício 16 da aula) Escreva uma função int contaConsoantes(char *str) que retorne a quantidade de consoantes presentes na string utilizando navegação por ponteiro.

25. **Extração de Vogais:**

(Baseado no Exercício 13 da aula) Escreva a função void extraiVogais(char *str, char *vetorVogais) que copie apenas as vogais da string str para o vetor vetorVogais utilizando ponteiros.

26. **Extração de Consoantes:**

(Baseado no Exercício 14 da aula) Escreva a função void extraiConsoantes(char *str, char *vetorConsoantes) que extraia apenas as consoantes da string original para um novo vetor.

27. **Tabela ASCII de uma String:**

Implemente a função void codigoAsc(char *p, int n) para percorrer uma string e exibir cada caractere acompanhado do seu respectivo valor numérico na tabela ASCII.

Bloco 6: Ponteiros para Ponteiros (Ponteiros Duplos **)

28. **Modificação via Ponteiro Duplo:**

(Baseado no Exercício 17 da aula) Declare int x = 10, int *px = &x e int **ppx = &px. Altere o valor de x para 100 utilizando unicamente o ponteiro para ponteiro ppx. Exiba o valor de x também usando ppx;

29. **Soma Dinâmica com Ponteiro Duplo:**

(Baseado no Exercício 18 da aula) Declare duas variáveis x e y. Crie um ponteiro int *p e, através do ponteiro de ponteiro int **pp = &p, aloque memória para um inteiro e armazene nele a soma de x e y.

30. **Troca de Endereços em Ponteiros Duplos:**

(Baseado no Exercício 19 da aula) Declare int x = 10, y = 20, com seus ponteiros px e py, e um ponteiro duplo ppx = &px. Faça *ppx = py e exiba o novo valor referenciado por *px e **ppx. Depois, altere **ppx = 50 e imprima os valores atualizados de x, y, *px e *py para observar o comportamento da memória.

Bloco 7: Exercícios Complementares (Vetores e Funções)

31.Soma dos Elementos de um Vetor:

Escreva uma função `int somaVetor(int *vt, int n)` que receba um vetor de inteiros e seu tamanho, e retorne a soma de todos os seus elementos calculada através de ponteiros.

32.Média Aritmética de um Vetor:

Crie a função `double mediaVetor(int *vt, int n)` que utilize a função do exercício anterior (`somaVetor`) para calcular e retornar a média aritmética dos elementos do vetor.

33.Contador de Números Pares:

Escreva a função `int contaPares(int *vt, int n)` que receba um vetor de inteiros e retorne a quantidade de números pares contidos nele.

34.Substituição de Valores Negativos:

Crie a função `void zeraNegativos(int *vt, int n)` que receba um vetor por referência e altere diretamente nele qualquer valor negativo para 0.

35.Multiplicação por Escalar:

Escreva uma função `void multiplicaVetor(int *vt, int n, int escalar)` que multiplique cada elemento do vetor por um número inteiro (escalar) informado pelo usuário.

36.Busca Sequencial:

Crie a função `int buscaElemento(int *vt, int n, int chave)` que busque por um determinado valor (chave) dentro do vetor. A função deve retornar o **índice** da primeira ocorrência do elemento ou -1 caso ele não seja encontrado.

37.Cópia de Vetor Dinâmico:

Escreva a função `int* clonaVetor(int *vt, int n)` que aloque dinamicamente um novo vetor com o mesmo tamanho N , copie todos os elementos de `vt` para ele e retorne o ponteiro para essa nova área de memória. *(Lembre-se de liberar com `free` na main!)*

38.União de Dois Vetores:

Crie uma função `int* concatenaVetores(int *v1, int n1, int *v2, int n2)` que receba dois vetores e seus respectivos tamanhos. A função deve alocar dinamicamente um novo vetor de tamanho $N1 + N2$, copiar os elementos de `v1` e `v2` em sequência e retornar o ponteiro para o novo vetor.

39.Compactação de Vetor (Remover Zeros):

Escreva uma função `int compactaVetor(int *vt, int n)` que remova todos os elementos iguais a 0 do vetor, deslocando os elementos válidos para a esquerda. A função deve retornar o **novo tamanho** do vetor.

40.Verificação de Vetor Ordenado:

Crie a função `int isOrdenado(int *vt, int n)` que verifique se os elementos de um vetor estão dispostos em ordem crescente. A função deve retornar 1 se o vetor estiver ordenado ou 0 caso contrário.